Лабораторная работа №1

Валеев Дильюс Рафисович, гр.4404

Задания на лабораторную работу:

Задание 1. Написать программу где каждый поток печатает свой идентификатор, количество потоков всего и строчку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Код:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
int main(int argc, char *argv[]) {
   int num_threads = 8;
   if (argc > 1) {
      num_threads = atoi(argv[1]);
   }
   omp_set_num_threads(num_threads);
   #pragma omp parallel
   {
      int tid = omp_get_thread_num();
      int total = omp_get_num_threads();
      printf("Hello World from thread %d of %d\n", tid, total);
   }
   return 0;
}
```

Вывод работы кода:

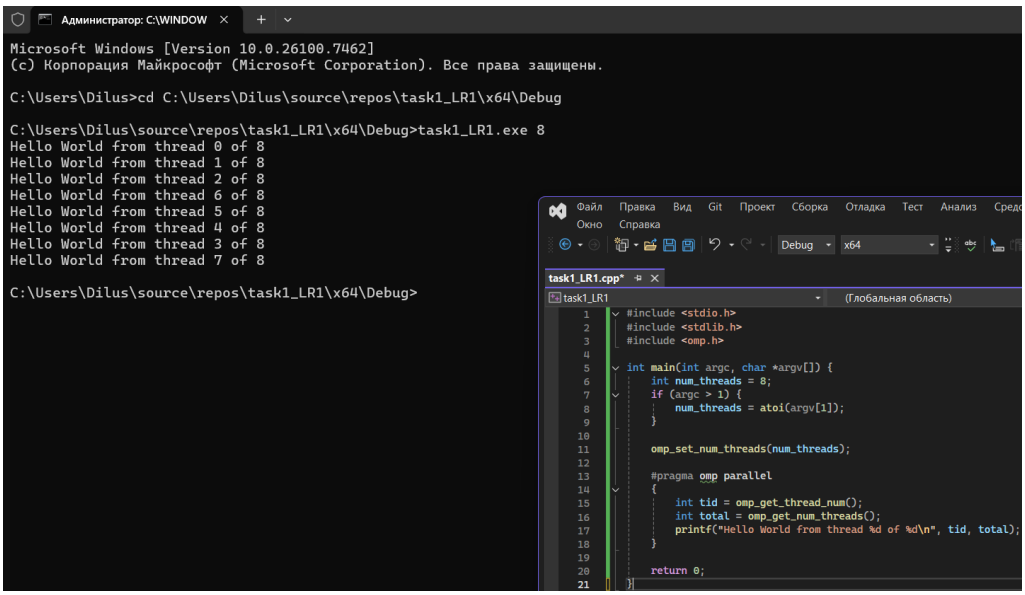


Рис. 1.

На рис. 1 показан первый запуск программы с 8 потоками. Видна команда запуска и полный вывод: каждый из 8 потоков напечатал свой идентификатор, общее число потоков и строку «Hello World».

Ответ на вопрос: Нет, вывод не всегда идентичен. Потоки в OpenMP выполняются асинхронно и параллельно. Порядок их переключения и выделения процессорного времени определяется планировщиком операционной системы. Если нет явной синхронизации, доступ потоков к стандартному потоку вывода print происходит в произвольном порядке. Если же задержка в потоке ничтожно мала, планировщик успевает запустить потоки по очередь.

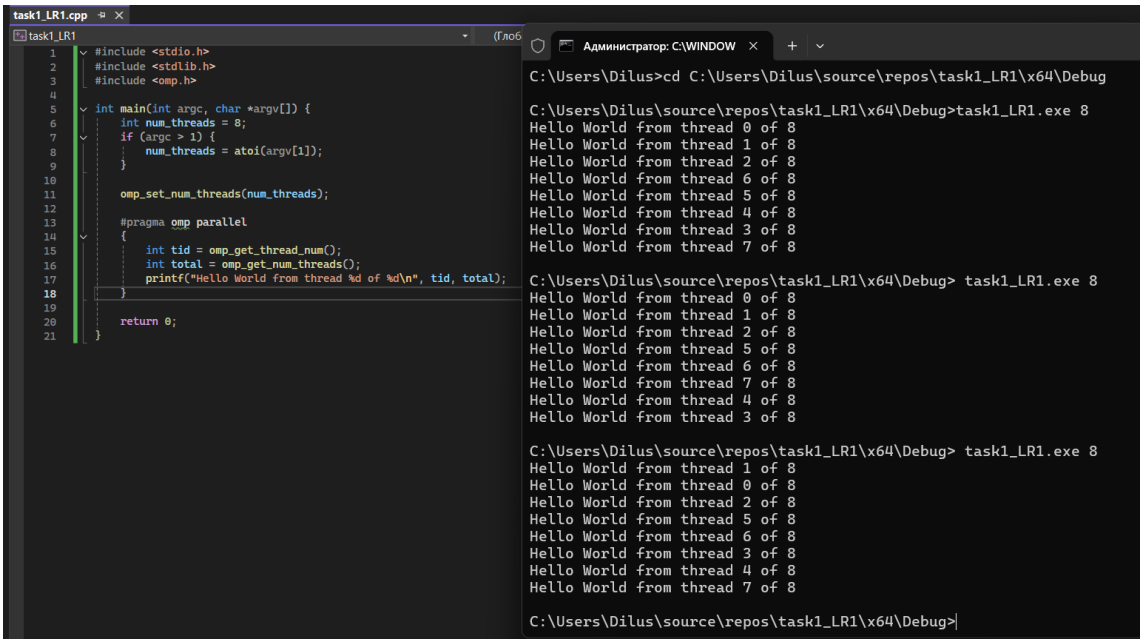


Рис. 2

На рис. 2 показан второй запуск той же команды. Порядок строк может отличаться от рисунка 1, так как порядок выполнения потоков не определён.

Задание 2. Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру.

Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива:

b[i] = (a[i-1] + a[i] + a[i+1])/3.0.

Запустите программу с 8-ю процессами при различных типах распределения работ.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>
void run_task2(int n, int threads, const char* schedule_type, int chunk) {
    double* a = (double*)malloc(n * sizeof(double));
    double* b = (double*)malloc(n * sizeof(double));
    if (!a || !b) {
        fprintf(stderr, "Memory allocation error\n");
        exit(1);
    }
    for (int i = 0; i < n; i++) {
        a[i] = (double)i;
        b[i] = 0.0;
    }
    omp_set_num_threads(threads);
    double start_time = omp_get_wtime();

    if (strcmp(schedule_type, "static") == 0) {
        if (chunk > 0) {
#pragma omp parallel for schedule(static, chunk)
            for (int i = 1; i < n - 1; i++) {
                b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
            }
        }
        else {
#pragma omp parallel for schedule(static)
            for (int i = 1; i < n - 1; i++) {
                b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
            }
        }
    }
    else if (strcmp(schedule_type, "dynamic") == 0) {
        int ch = (chunk > 0) ? chunk : 1;
#pragma omp parallel for schedule(dynamic, ch)
        for (int i = 1; i < n - 1; i++) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
        }
    }
    else if (strcmp(schedule_type, "guided") == 0) {
        if (chunk > 0) {
#pragma omp parallel for schedule(guided, chunk)
            for (int i = 1; i < n - 1; i++) {
                b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
```

Рис. 3

```
        else {
#pragma omp parallel for schedule(guided)
            for (int i = 1; i < n - 1; i++) {
                b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
            }
        }
    }
    else if (strcmp(schedule_type, "runtime") == 0) {
#pragma omp parallel for schedule(runtime)
        for (int i = 1; i < n - 1; i++) {
            b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0;
        }
    }
    else {
        printf("Unknown schedule type: %s\n", schedule_type);
        free(a);
        free(b);
        return;
    }
    double end_time = omp_get_wtime();

    printf("Schedule: %-8s | Threads: %d | N: %d | Time: %.6f sec | Check b[1]=%.2f, b[%d]=%.2f\n",
        schedule_type, threads, n, (end_time - start_time), b[1], n - 2, b[n - 2]);
    free(a);
    free(b);
}
int main(int argc, char* argv[]) {
    int n = 16000;
    int threads = 8;
    char schedule_type[32] = "all";
    int chunk = 0;
    if (argc > 1) threads = atoi(argv[1]);
    if (argc > 2) n = atoi(argv[2]);
    if (argc > 3) strncpy(schedule_type, argv[3], sizeof(schedule_type) - 1);
    if (argc > 4) chunk = atoi(argv[4]);
    if (strcmp(schedule_type, "all") == 0) {
        printf("=== Testing OpenMP Work Schedules ===\n");
        run_task2(n, threads, "static", 0);
        run_task2(n, threads, "static", 100);
        run_task2(n, threads, "dynamic", 1);
        run_task2(n, threads, "dynamic", 100);
        run_task2(n, threads, "guided", 10);
        run_task2(n, threads, "runtime", 0);
    }
    else {
        run_task2(n, threads, schedule_type, chunk);
    }
```

Рис. 4

Программа выделяет в динамической памяти два массива типа *double* размером $N = 16\,000$ элементов: исходный a и результирующий b.

Массив a инициализируется значениями, равными порядковому индексу: $a[i] = i$.

С помощью вызова *omp_set_num_threads(threads)* задаётся количество рабочих потоков (8 потоков).

Распараллеливание вычислений выполняется директивой *#pragma omp parallel for* с различными условиями распределения итераций *(schedule): static, dynamic, guided и runtime.*

Для всех внутренних элементов (от $i = 1$ до $n - 2$, исключая крайние) вычисляется локальное среднее по формуле:

$$b[i] = \frac{a[i-1] + a[i] + a[i+1]}{3.0}$$

Вывод работы кода:

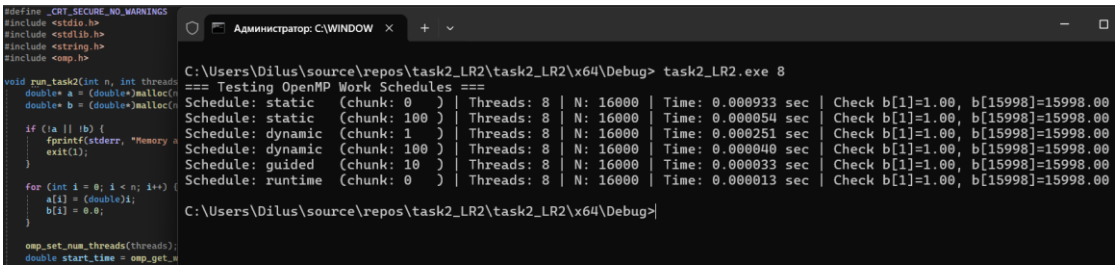


Рис. 5

static: (статическое): Пространство итераций делится на равные блоки на этапе запуска и жёстко закрепляется за потоками. Минимальные накладные расходы. Для нашей задачи оно оптимально, так как вычислительная сложность каждой итерации одинакова.

dynamic: (динамическое): Итерации нарезаются на порции (chunk). Поток, завершив порцию, запрашивает у рантайма следующую. Эффективно при неравномерной нагрузке, но даёт накладные расходы на синхронизацию очереди задач.

guided: (экспоненциально убывающее): Порции итераций сначала выдаются большими блоками, затем размер блока уменьшается. Компромисс между static и dynamic.

runtime: Конкретный тип и размер блока не зашиты в бинарник, а определяются во время запуска через системную переменную окружения OMP_SCHEDULE.

Задание 3. Модифицируйте задачу 1 так, чтобы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения. Постарайтесь найти как можно больше.

Коды:

Способ 1. Активное ожидание флага с принудительной синхронизацией памяти (Spin-wait + #pragma omp flush)

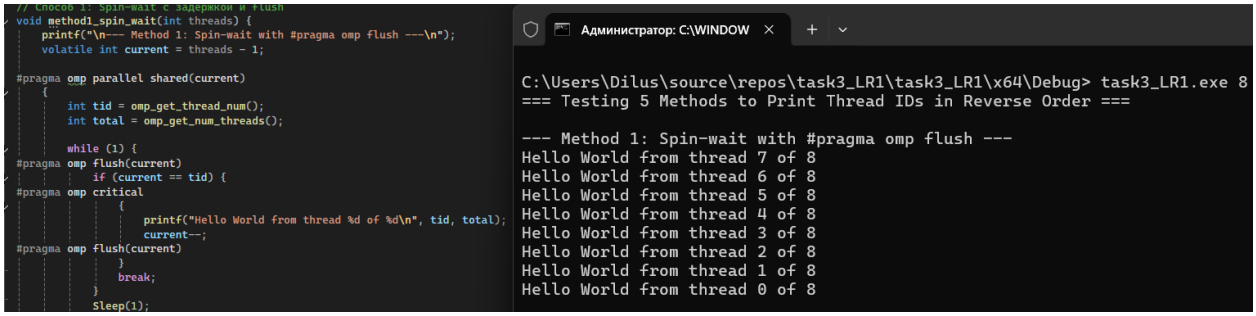


Рис. 6

Принцип работы: Потоки параллельно опрашивают разделяемую переменную очереди current. Директива #pragma omp flush принудительно синхронизирует значение этой переменной между локальными кэшами ядер и оперативной памятью, гарантируя, что потоки видят актуальное значение. Когда наступает очередь потока (current == tid), он входит в критическую секцию #pragma omp critical, печатает свой номер, уменьшает значение current на единицу и завершает цикл.

Особенности: Низкоуровневый алгоритм на разделяемой памяти. Требует добавления микропаузы (Sleep), чтобы потоки не монополизировали процессорные кванты и не вызывали голодание старших потоков в планировщике ОС.

Способ 2. Пошаговая барьерная синхронизация (#pragma omp barrier)



Рис. 7

Принцип работы: Все потоки одновременно выполняют цикл с числом итераций, равным общему числу потоков, где шаг цикла убывает от total - 1 до 0. На каждом шаге условие if (tid == step) становится истинным ровно для одного потока — он выполняет печать в консоль. Сразу после этого директива

#pragma omp barrier блокирует выполнение всех потоков до тех пор, пока каждый из них не достигнет этой точки синхронизации. Только после сбора всех потоков на барьере начинается следующий шаг цикла.

Особенности: Обеспечивает строгий детерминированный порядок без использования мьютексов и критических секций, опираясь исключительно на примитив барьера стандарта OpenMP.

Способ 3. Встроенное упорядочивание итераций цикла (#pragma omp ordered)

```
// Способ 3: Директива упорядочивания (#pragma omp ordered)
void method3_ordered(int threads) {
    printf("\n--- Method 3: Ordered directive (#pragma omp ordered) ---\n");

#pragma omp parallel
    {
#pragma omp for ordered schedule(static, 1)
        for (int i = 0; i < threads; i++) {
#pragma omp ordered
            {
                int target_tid = threads - 1 - i;
                printf("Hello World from thread %d (target ID: %d of %d)\n",
                    omp_get_thread_num(), target_tid, omp_get_num_threads());
```

```
--- Method 3: Ordered directive (#pragma omp ordered) ---
Hello World from thread 0 (target ID: 7 of 8)
Hello World from thread 1 (target ID: 6 of 8)
Hello World from thread 2 (target ID: 5 of 8)
Hello World from thread 3 (target ID: 4 of 8)
Hello World from thread 4 (target ID: 3 of 8)
Hello World from thread 5 (target ID: 2 of 8)
Hello World from thread 6 (target ID: 1 of 8)
Hello World from thread 7 (target ID: 0 of 8)
```

Рис. 8

Принцип работы: Используется параллельный цикл #pragma omp for ordered. Директива #pragma omp ordered гарантирует, что блок кода внутри неё будет выполняться потоками строго последовательно, в естественном порядке возрастания индекса цикла i = 0, 1, 2.... Чтобы развернуть вывод в обратную сторону, внутри блока вычисляется инвертированный номер: target_tid = threads - 1 - i.

Особенности: Высокоуровневая встроенная конструкция OpenMP. Позволяет организовать строгую очерёдность без ручного управления счётчиками и барьерами.

Способ 4. Синхронизация через объект блокировки OpenMP (omp_lock_t).

```
// Способ 4: Корректное использование замков (omp_lock_t) с передачей готовности
void method4_locks(int threads) {
    printf("\n--- Method 4: Lock-protected turn indicator (omp_lock_t API) ---\n");

    omp_lock_t lock;
    omp_init_lock(&lock);
    int current_turn = threads - 1;

#pragma omp parallel shared(lock, current_turn)
    {
        int tid = omp_get_thread_num();
        int total = omp_get_num_threads();
        int done = 0;

        while (!done) {
            omp_set_lock(&lock);
            if (current_turn == tid) {
                printf("Hello World from thread %d of %d\n", tid, total);
                current_turn--;
                done = 1;
            }
            omp_unset_lock(&lock);

            if (!done) {
                Sleep(1);
            }
        }
    }

    omp_destroy_lock(&lock);
}
```

```
--- Method 4: Lock-protected turn indicator (omp_lock_t API) ---
Hello World from thread 7 of 8
Hello World from thread 6 of 8
Hello World from thread 5 of 8
Hello World from thread 4 of 8
Hello World from thread 3 of 8
Hello World from thread 2 of 8
Hello World from thread 1 of 8
Hello World from thread 0 of 8
```

Рис. 9

Принцип работы: Используется программный интерфейс замков (мьютексов) библиотеки OpenMP: omp_init_lock, omp_set_lock, omp_unset_lock и omp_destroy_lock. Доступ к общей переменной очереди current_turn защищается замком lock. Поток захватывает замок через omp_set_lock, проверяет условие current_turn == tid, в случае совпадения выводит строку и декрементирует переменную, после чего освобождает замок через omp_unset_lock.

Особенности: Корректная модель владения замками: каждый поток захватывает и освобождает замок в пределах своего контекста исполнения, что предотвращает ошибки типа Deadlock и User Error: lock not owned в рантайме OpenMP.

Способ 5. Разделение фаз вычислений и вывода (Буферизация)



Рис. 10

Принцип работы: Алгоритм разделяется на две независимые фазы:

Параллельная фаза: Каждый поток независимо и без точек синхронизации или блокировок сохраняет свой идентификатор в персональную ячейку общего массива buffer[tid] = tid.

Последовательная фаза: После завершения параллельной области основной поток обходит заполненный массив в обратном направлении (от threads - 1 до 0) и выводит сохранённые данные на экран.

Особенности: Самый эффективный архитектурный подход: полностью устраняет гонку потоков за стандартный поток вывода (stdout) и накладные расходы на постоянную синхронизацию в процессе параллельной работы.

Ссылка на репозиторий в Github: https://github.com/ValeevDiL/ACTS-labs/tree/6b930b4b5ea0da29968253c8a99282217db9a63e/lab1